2IP90
Lecture 2

Input/Output (Chapter 6)

Types (Ch. 2-4)

Strings (Ch. 3)

switch

Programming

```java
import java.util.Scanner;

class Interest {
    Scanner scanner = new Scanner(System.in);

    public void calcInterest() {
        double saldo;  // aka balance
        double rate;  // percentage

        saldo = 250;
        System.out.println("Give interest rate as
        rate = scanner.nextDouble();

        System.out.println("After a year you get
                           (saldo * rate / 100));
    }

    public static void main(String[] args) {
        ( new Interest() ).calcInterest();
    }
}
```

instructions to a computer

```java
                mars1, (int)(gschaal * 6800), (int)(aschaal * 1.524),
                0, PI2/687, Color.red);

        CelestialBody[] sun1 = {aarde, venus, mars, mercurius};
        sun = new CelestialBodyExtra(sun1, 50, 0, 0, 0, Color.yellow);

        setBackground(Color.black);

        timer = new Timer(20, this);
    }

    public void start() {
        timer.start();
    }

    public void stop() {
        timer.stop();
    }

    // sets timer interval, and with that the speed of the planets
    // 1 timer step moves everything the equivalent of timestep days
    // so delay : (timestep*24*3600+1000) = simulation time : simulated
    public void setDelay(int delay) {
        timer.setDelay(delay);
    }

    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == timer) {
        // called by timer at regular intervals
            sun.step(tijdstap);
            repaint();
        } else if (e.getActionCommand().equals("Start")) {
            this.start();
        } else if (e.getActionCommand().equals("Stop")) {
            this.stop();
        }
    }
```

⇒ how does a computer work?

Parts of a computer

- several input/output devices
  display, keyboard, disk, network, ...
- CPU (processor): the thing that computes
- memory (RAM, *werkgeheugen*)

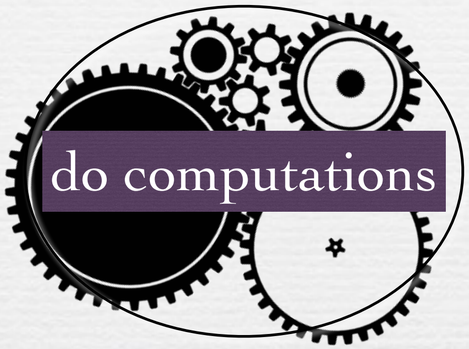
How does it work?



Processor

Memory cells

store
(write)



fetch
(read)

input/output devices

keyboard
mouse
network
screen
disk

Only a calculator?

- Processor has a limited set of *instructions* fixed in the electronic circuits

  How to perform complex tasks beyond the limited set of instructions?

- Instructions are made for computers, not for humans, e.g.,
  read memory cell 12345, add 1 to it, and write it back

Two Tricks

- The program trick: store long sequences of instructions in memory
- The translation trick: translate human readable programs *automatically* into sequences of machine readable instructions
- In Java
  storing a value is by assignment (x = 1)
  fetching a value is by using in expressions (x + 3)

Types of rules

Syntax - Semantics - Conventions

1. **Syntax**: grammar
   if (...)
2. **Semantics**: meaning
   e.g., "when guard is true, execute first part"
3. **Conventions**: wise advise
   indent after each {

   includes checkstyle

when not observed

1. **Syntax**: program does not run
2. **Semantics**: program runs, but with wrong results
3. **Conventions**: program runs, results not affected, but difficulties in understanding and maintaining; bad grades, bad reviews

Output

when both are followed by
System.out.println(" and y is "+ y);
the result is:

x is 3
and y is 4

resp.

x is 3 and y is 4

- console output:
  - System.out.println("x is "+x);
  - without new line: System.out.print("x is "+x);
  - numeric arguments are automatically converted to their textual decimal representation (using a decimal point)

not used in this course

x = 3
y = 4

- dialog output: JOptionPane.showMessageDialog(null, "boe!");

Input

- for console input use Scanner:
  - import java.util.Scanner;
  - Declare variable: Scanner scanner; (or any other name instead of scanner)
  - Create and connect object: scanner = new Scanner(System.in);
  - scanner.hasNext() is true when there is input left (always true in interactive situation, not always true when reading from file (such as in Peach)
  - scanner.hasNextInt(), hasNextDouble() etc. are true when such a number is on top of input
- dialog input: JOptionPane.showInputDialog(null, "type wat");

terminal in VS Code

not used in this course

Input with Scanner

- next() reads next word (whitespace separated) and gives a String, without surrounding whitespace
- nextInt() ditto, gives an int
- nextDouble(), nextBlah() ditto, gives a double (or type Blah)
- hasNext() is true if there is still more than whitespace on input (never false when reading from keyboard)
- hasNextInt() is true if there is still a word on input and that word is an integer number
- hasNextDouble(), hasNext.. ditto, gives a double (or type ...)
- NB: what separates the words (tokens) can be changed. Default is whitespace (spaces, tabs, newlines).

Types

- type: a set of values, such as
  - int (more or less $\mathbb{Z}$, the integer numbers)
  - double (more or less $\mathbb{R}$, the real numbers)
  - bool ({true, false})
  - more

Types

- every expression and variable has a type
  - known before execution ("at compile time") → called static typing
- types must match → called type safety
  - equal or compatible
- compatible types: "smaller" fits in "larger"
  - e.g., `double x = 3;`



numeric types

- integer types: int, long (byte, short, char)
- floating point types: float, double

The devious division

- som/3 doesn't work when som is an int
  Why?

  There are two versions of /

  1. when one of the arguments is a double:
     ordinary division
     e.g., 7.0 / 2 == 3.5

  2. when both arguments are int:
     integer division

     e.g., 7 / 2 == 3

- integer division can be handy in programming
  still, having one symbol for two operations is confusing

text types

- char: 'a' 'b' '0'
- String: "abc" "a" ""
  "sir, I bear a rhyme excelling in mystic force and magic spelling"

String operations

- +: concatenation
  "good" + "bye" ➳ "goodbye"
- equals: equality
  "good".equals("bad") ➳ false
- compareTo: lexicographic ordering
  "big".compareTo("huge") ➳ a negative number
  "huge".compareTo("big") ➳ a positive number
  "equal".compareTo("equal") ➳ 0
- length: gives number of characters
  "five".length() ➳ 4
- note: == should not be used for comparing Strings
- == on Strings is allowed, but doesn't what you want, it produces false in most cases



String operations (ctd)

- substring: gives subsequence by position
  "Building".substring(1,4) ➳ "uil"

first character has position 0

- indexOf: gives position of substring
  "Building".indexOf("uil") ➳ 1
  "Building".indexOf("valk") ➳ -1

String.format

- for convenient printing (using interpolation)
  - String.format("x=%d, y=%d, z=%d", x, y, z)
    ⇒ "x=1, y=10, z=-3" (depending on values of x,y,z)
- for formatting of numbers (and dates)
  - String.format("temperature is %.1f ºC", t)
    ⇒ "temperature is 13.1 ºC"
- alternative notation:
  - "temperature is %.1f ºC".formatted(t)

String.format specifiers

specifier	applies to	output
%s	any type	String value
%d	integer	decimal representation
%f	float, double	decimal representation
%e	float, double	scientific notation

String.format extra specifiers

extra	applies to	effect
%nX	X any specifier	width: use n positions
%.nX	X a floating point spec	use n decimals
%-nX	X any specifier	width n, left aligned
%n.mX	X a floating point spec	m decimals, total width n

String.format examples

- for convenient printing (using interpolation)
  - club1 = “PSV”;
    club2 = “Ajax”;
    String.format("%s is better than %s", club1, club2)
    ⇒ "PSV is better than Ajax"
- String.format(“<%.8f>”, Math.PI) ⇒ <3.14159265>
- String.format(“<%8.3f>", Math.PI) ⇒ <   3.142>

can be any String expression

Yes!
otherwise we would see 3.141

does format round the result?

Random numbers

- Rolling dice, flipping coins, drawing numbers. For games, simulations, etc.
- (pseudo) random number generator built in in Java
- not truly random, it is a deterministic, but complex calculation from the previous number
- for truly random numbers (from, e.g., atmospheric noise) there are websites and webservices

Random number

- its use is similar to Scanner
- import java.util.Random;
- Random random = new Random();
- int n;
  double x;
- n = random.nextInt(10); // puts a random int value in [0..10) into n
- x = random.nextDouble(); // puts a random double value in [0..1) into x

Random() is not really random

- next number is calculated with a formula such as (37 * x + 13) % 97
  where the first x is derived from, e.g., the time
- pretty random, but deterministic. Same starting number gives you the same sequence (which can be useful!)
- to have real random numbers, use a physical process (or quantum mechanics) (available in hardware or via webservices such as random.org)

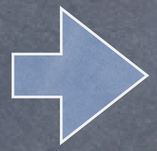
Seeding

- provide the starting number yourself
- Random random = new Random(1357);
- creates a random number generator with given starting point (the seed). Sequence is the same if the provided number is the same.
- useful for debugging etc.

switch

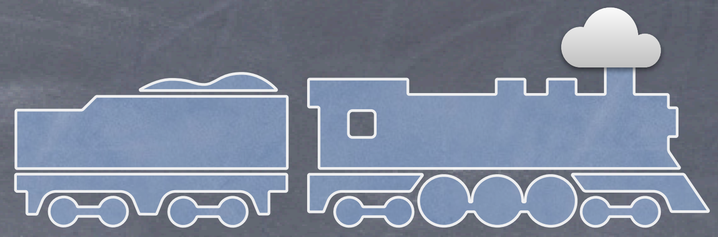
- alternative to repeated if-construction on the value of a variable

```
if (x == 0) {
  blah
} else if (x == 1) {
  yada
} else if (x == 7) {
  yada
} else {
  et cetera
}
```



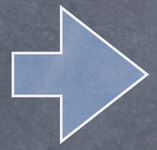
```
switch(x) {
    case 0 -> blah
    case 1 -> yada
    case 7 -> yada
    default -> et cetera
}
```

switch - ancient syntax



- alternative to repeated if-construction on the value of a variable

```
if (x == 0) {
  blah
} else if (x == 1) {
  yada
} else if (x == 7) {
  yada
} else {
  et cetera
}
```



```
switch(x) {
   case 0: blah
           break;
   case 1:
   case 7: yada
           break;
   default: et cetera
}
```

this is the advantage of the break system

where does your program start

- general way in Java:
  public static void main(String[] a) {  }
- methods with label static have some limitations (more about that later), but are very good for start up code
- therefore we often put the code of the program in the main
- or... (next slide)

where does your program start (bis)

- or we put the program code in a non-static method and call this in the main method:

```
class X {
  void demo( ) {
     THE PROGRAM
  }

  public static void main(String[] a) {
     new X().demo();
  }
}
```

- new X() creates an object of the class X
- this allows the call of the non-static method demo
  (non-static methods require an object to run on)
- more about this later

where does your program start

- general way in Java:
  public static void main(String[] a) {  }
- NetBeans allows more possibilities
  - static void main() {  }
  - void main() {  }